

README – Système de Chronométrage Automatique Multi-Voitures

1. Présentation générale

Ce projet implémente un système complet de chronométrage automatisé pour des courses de voitures autonomes ou mini-voitures.

Il combine plusieurs technologies :

- Détection de passage via capteurs **VL53L0X** (distance)
- Multiplexage via **TCA9548A** pour gérer plusieurs capteurs ayant la même adresse I2C
- Lecture de **QR codes** via une caméra USB
- Communication **XBee** pour envoyer les signaux START et END
- Archivage automatique des résultats (CSV + photos)
- Détection automatique du **sens de passage**
- Gestion simultanée de plusieurs voitures

Le système est conçu pour fonctionner sur un **Raspberry Pi**.

2. Matériel requis

- Raspberry Pi avec I2C activé
- Caméra USB (index 0 par défaut)
- Module XBee connecté en USB
- Multiplexeur I2C TCA9548A (adresse 0x70)
- Plusieurs capteurs VL53L0X
 - 2 pour la détection du sens
 - 5 pour la détection verticale des voitures

3. Installation

Dépendances Python

```
pip install opencv-python pyzbar adafruit-circuitpython-vl53l0x
```

Paquets système (Debian / Raspberry Pi OS)

```
sudo apt update  
sudo apt install libzbar0 python3-opencv
```

4. Fichiers générés

À chaque course, un dossier est créé automatiquement dans :

/home/pi/capture_covaciel

Exemple :

```
capture_covaciel/  
├── 23-04-2026_15-00-00/  
│   ├── resultats.csv  
│   ├── voiture_12_tour_1.jpg  
│   ├── voiture_12_tour_2.jpg  
│   └── ...
```

Le fichier *resultats.csv* contient :

- *voiture_id*
- *tour*
- *temps_tour*
- *temps_total*
- *timestamp*
- *photo*

5. Configuration (dans le script)

Les paramètres principaux sont regroupés en haut du fichier Python :

- *USB_CAMERA_INDEX* : index de la caméra (0 = /dev/video0)
- *CHANNEL_LIGNE_GAUCHE* / *CHANNEL_LIGNE_DROITE* : capteurs pour le sens
- *CHANNEL_VERTICAUX* : capteurs de détection des voitures
- *SEUIL_LIGNE* / *SEUIL_VERTICAL* : seuils de détection (en mm)
- *XBEE_PORT* / *XBEE_BAUD* : configuration du module XBee
- *BASE_DIR* : dossier d'archivage

Ces valeurs peuvent être adaptées selon l'installation matérielle.

6. Lancement

1. Brancher tout le matériel
2. Lancer le script :

python chrono.py

3. Le script demande le nombre de tours
4. Le signal START est envoyé automatiquement via XBee
5. La détection commence immédiatement

7. Diagnostic et dépannage

Au démarrage, le script affiche :

- Présence de la caméra
- Présence des capteurs ligne
- Présence des capteurs verticaux
- Liste des canaux actifs

Problèmes courants

Caméra non détectée

→ Vérifier `/dev/video0`

→ Tester avec : `v4l2-ctl --list-devices`

Capteur absent

→ Vérifier câblage SDA/SCL

→ Vérifier alimentation 5V

→ Vérifier adresse du TCA9548A

QR non détecté

→ Vérifier la luminosité

→ Vérifier la netteté de la caméra

→ Vérifier que le QR est centré

XBee non détecté

→ Vérifier `/dev/ttyUSB0`

→ Vérifier les permissions du port série

8. Points techniques importants

Thread caméra

La caméra tourne dans un thread séparé pour éviter de bloquer la boucle principale.

La variable `last_frame` contient toujours la dernière image valide.

Multiplexeur TCA9548A

Un seul canal peut être actif à la fois.

Le script active un canal avec `select_channel(ch)` puis attend 2 ms pour stabiliser le capteur.

Lecture QR robuste

Le script teste plusieurs traitements :

- image grise
- seuillage Otsu
- flou gaussien

Cela améliore la lecture même en conditions difficiles.

Communication XBee

Les messages START et END sont envoyés plusieurs fois pour garantir la réception radio.

9. Fin de course

Lorsque toutes les voitures ont terminé :

- Le gagnant est déterminé automatiquement
- Le classement final est affiché
- Le signal END est envoyé via XBee
- La caméra est arrêtée proprement